Extra Topic for Luxair

None

Version 1.0, March 2026

Table of Contents

Chapter 1: Django REST Framework Basics . . . 1
- About Django REST framework . . . 2
- Django REST Framework Architecture . . . 3
- Initial setup . . . 5
- Serializing the hard way . . . 6
- Serializing the easier way . . . 7
- Implementing RESTful views . . . 9
- Configuring RESTful routes . . . 11
- Class-based Views . . . 14

Chapter 2: Django REST Viewsets . . . 17
- What are viewsets? . . . 18
- Creating Viewsets . . . 19
- Setting up routes . . . 21
- Customizing viewsets . . . 23
- Adding pagination . . . 24

Chapter 3: Django REST Filters . . . 26
- What are filters? . . . 27
- Creating filters . . . 28
- Filter shortcuts . . . 30
- Using filters . . . 31

Chapter 4: Django REST Documentation . . . 33
- Documentation? . . . 34
- Generating an OpenAPI schema: Part I . . . 35
- Generating an OpenAPI schema: Part II . . . 36

Chapter 5: Unit Testing with pytest . . . 38
- What is a unit test? . . . 39
- The pytest module . . . 40
- Creating tests . . . 42
- Running tests (basics) . . . 44
- Special assertions . . . 45
- Fixtures . . . 48
- User-defined fixtures . . . 49
- Builtin fixtures . . . 52
- Configuring fixtures . . . 56
- Parametrizing tests . . . 59
- Marking tests . . . 62

Running tests (advanced) . . . 64
Making test files executable . . . 66
Skipping and failing . . . 67
Mocking data . . . 70
Mocking in pytest . . . 71
Mock where the object is used . . . 72
Pytest plugins . . . 79
Index . . . 81

Chapter 1: Django REST Framework Basics

Objectives

- Learn basics of the Django REST framework
- Define serializers for models
- RESTful routing
- Create function-based API views
- Create class-based API views

This chapter assumes a Django project is already created, with some available models.

About Django REST framework

- Flexible package for building REST APIs
- Includes OAuth authentication
- Supports ORM and non-ORM data
- Very customizable

The Django REST framework is a comprehensive package for creating RESTful APIs. It leverages the existing Django configuration and infrasructure.

The basic idea is to provide serializers that transform your models (or other data) into JSON (or possibly other formats) that can be returned via your application's REST API.

REST Framework provides class-based views, viewsets, and filters that make it easy to create REST apps.

A big feature of REST Framework is the browsable API.

REST Framework includes OAuth and other kinds of authentication, and supports both ORM and non-ORM data sources. It is extremely customizable.

© 2026 JS Consulting *(Licensed to Webucator)*

Django REST Framework Architecture

- Serializers
- Filters
- Class-based views
- Viewsets

The crux of REST Framework, like a page-based Django app, is the model. REST Framework uses normal Django models. What it provides beyond that are serializers, filters, class-based views (CBVs), and viewsets. These work together to let you build apps with less coding.

For each model, you define a serializer. This is a class that converts the data in the object to native Python datatypes that can then be rendered into JSON (or some other format). In the serializer, you can control which fields are exposed to the API as well as performing conversions, etc.

Each model can also have a *filter*. This is a class that decribes query strings for the model. Filters allow you to fetch data with a URL like `…/api/contacts?name=Fred`.

To save writing a large number of similar view functions, REST Framework provides two kinds of classes that abstract away common tasks.

The first kind of classes are API views, which provides responses to HTTP requests such as GET or POST. They save the trouble of writing individual view functions for each different HTTP request. There are several basic class-based API views, plus mixins for customization.

The second kind are *viewsets*. Viewset further abstract the data from individual view functions. They work in conjunction with *router*, to automatically set up URL paths.

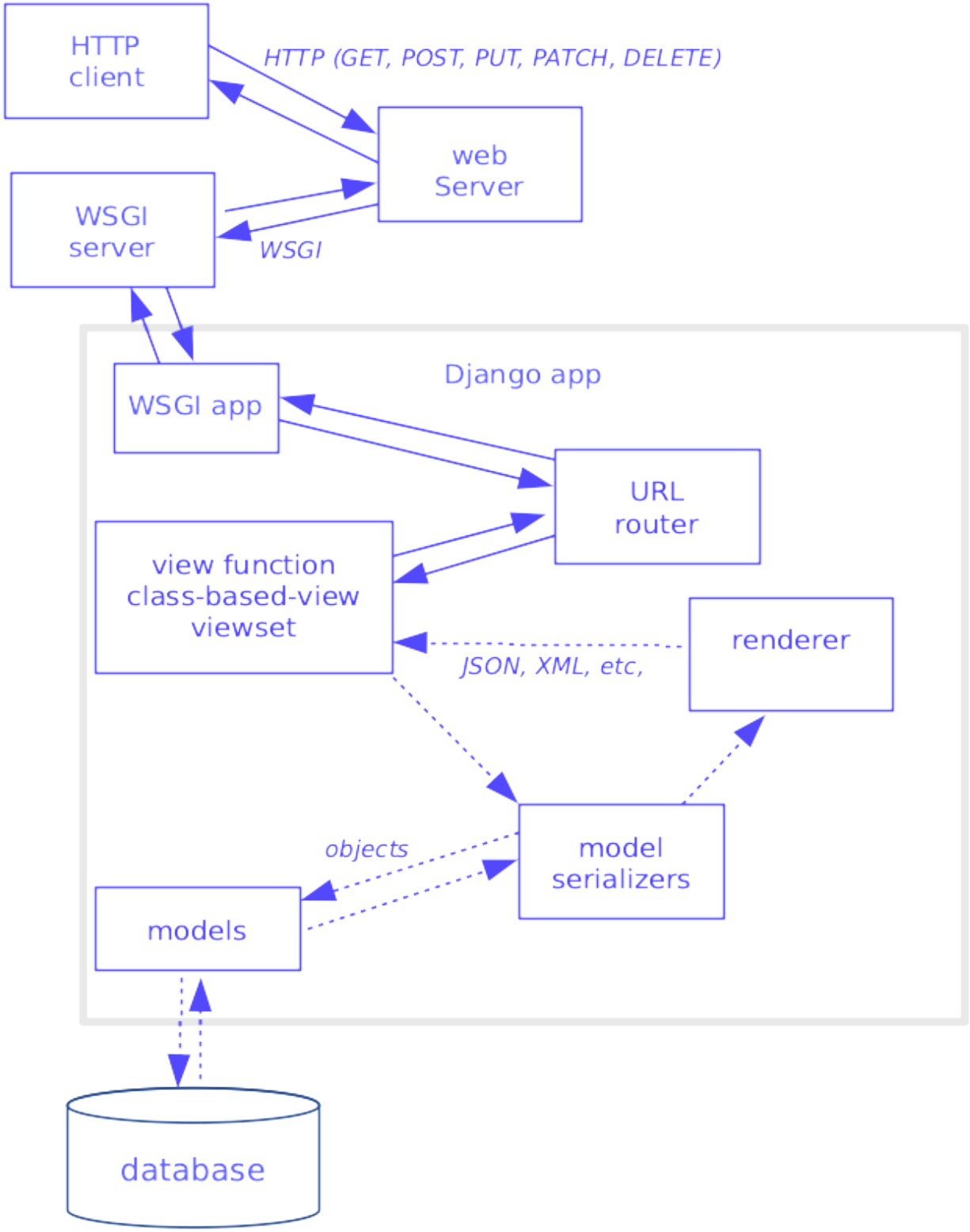

Figure 1. Django REST Architecture

© 2026 JS Consulting *(Licensed to Webucator)*

Initial setup

- Add Django REST Framework to settings.py
- Create models
- Create serializers

To get started add **rest_framework** to INSTALLED_APPS in settings.

add a configuration dictionary to the project's settings.py file. This will contain all of the global settings for the REST API.

Serializing the hard way

- Use generic Serializer
- Specify all fields
- Define create and update methods

One way to create serializers is to start with a class that inherits from rest_framework.serializers.Serializer. Then add class variables that map to the fields in your models, or at least all the fields you want to expose in in your REST app.

Once this is done, the serializer class can be used with the matching model to serialize and deserialize the data.

To test serializers, they can be used in the Django shell:

```
In [20]: from contacts_core.models import City, Contact
In [21]: contact = Contact.objects.all().first()
In [22]: contact
Out[22]: <Contact: Contact object (99a5ba00-65bb-4ec0-8400-a7876fe6155c)>
In [23]: serializer = ContactSerializerPlain(contact)
In [24]: serializer.data
Out[24]: {'id': '99a5ba00-65bb-4ec0-8400-a7876fe6155c', 'first_name': 'John',
'last_name': 'Strickler', 'street_address': '4110 Talcott Dr.', 'postcode': '27705',
'dob': '1956-10-31', 'city': OrderedDict([('id', '7dc9cfb0-243f-46ea-b9a1-f1e29807308f'),
('name', 'Durham'), ('admindiv', 'NC'), ('country', 'US')])}
In [25]: from rest_framework.renderers import JSONRenderer
In [27]: json_data = JSONRenderer().render(serializer.data)
In [29]: json_data
Out[29]: b'{"id":"99a5ba00-65bb-4ec0-8400-
a7876fe6155c","first_name":"John","last_name":"Strickler","street_address":"4110 Talcott
Dr.","postcode":"27705","dob":"1956-10-31","city":{"id":"7dc9cfb0-243f-46ea-b9a1-
f1e29807308f","name":"Durham","admindiv":"NC","country":"US"}}'
```


© 2026 JS Consulting *(Licensed to Webucator)*

Serializing the easier way

- Use ModelSerializer
- Reads metadata from models
- Only needs nested Meta class

The easy way to create serializers is to let the ModelSerializer class do all the work. It can read the metadata from your models and generate the serializer fields.

Example

djangoexamples/contacts/contacts_core/api/serializers.py

```
from rest_framework import serializers
from contacts_core.models import City, Contact


class AnimalSerializer(serializers.Serializer):

    animal = serializers.CharField(max_length=32)
    country = serializers.CharField(max_length=32)

class CitySerializerPlain(serializers.Serializer):

    id = serializers.UUIDField()
    name = serializers.CharField(max_length=32)
    admindiv = serializers.CharField(max_length=32)
    country = serializers.CharField(max_length=2)

class ContactSerializerPlain(serializers.Serializer):

    id = serializers.UUIDField()
    first_name = serializers.CharField(max_length=32)
    last_name = serializers.CharField(max_length=32)
    street_address = serializers.CharField(max_length=32)
    postcode = serializers.CharField(max_length=16)
    dob = serializers.DateField()
    city = CitySerializerPlain()

class CitySerializer(serializers.ModelSerializer):
    class Meta:
        model = City
        fields = ('id', 'name', 'admindiv', 'country')

class ContactSerializer(serializers.ModelSerializer):
    city = serializers.HyperlinkedRelatedField(view_name='contacts_core:api:city-detail',
read_only=True)

    class Meta:
        model = Contact
        fields = ('id', 'first_name', 'last_name', 'street_address', 'city', 'postcode',
'dob')
```


© 2026 JS Consulting *(Licensed to Webucator)*

Implementing RESTful views

- Define JSON response from HttpResponse
- Import csrf_exempt decorator
- Create normal views
- Return JSON rather than HTML

For really simple cases, you can create more or less normal Django function-based views. However, you'll want to decorate them with the csrf_exempt decorator which protects them from cross-site scripting.

And of course the views should return JSON, not HTML, so use the serializers that were created earlier plus the JSON renderer.

Example

djangoexamples/contacts/contacts_core/api/fb_views.py

```
# not needed for REST CBVs or Viewsets
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework.renderers import JSONRenderer
from contacts_core.models import Contact
from .serializers import ContactSerializerPlain

# Create your RESTful views here.

# example without template (only used in class -- always use templates in real life):
@api_view(['GET'])
def hello(request):
     message = {"message": "Welcome to Contacts API Core"}
     renderer = JSONRenderer()
     return Response(renderer.render(message), 200)

@api_view(['GET'])
def contacts(request):
     contacts = Contact.objects.all()
     serializer = ContactSerializerPlain(contacts, many=True)
     contacts_json = JSONRenderer().render(serializer.data)
     return Response(contacts_json, 200)

@api_view(['GET'])
def contacts_detail(request, pk):
     contacts = Contact.objects.filter(id=pk)
```

```
    serializer = ContactSerializerPlain(contacts)
    contacts_json = JSONRenderer().render(serializer.data)
    return Response(contacts_json, 200)
```


© 2026 JS Consulting *(Licensed to Webucator)*

Configuring RESTful routes

- Add views as normal to *app*/urls.py
- Allow for variable parts of URL
- Use named regular expression groups.

Add route configuration to the app's urls.py, and register them in the project's urls.py.

Example

djangoexamples/contacts/contacts/urls.py

```
from django.conf import settings
from django.urls import path, include
from django.contrib import admin

urlpatterns = [
    path('admin', admin.site.urls),
    # path('dogs', include('contacts_core.urls', )),
    # path('art', include('contacts_core.urls', )),
    path('', include('contacts_core.urls', )),
    path('auth/', include('djoser.urls')),
    path('auth/', include('djoser.urls.authtoken')),
]

# include Django Debug toolbar if DEBUG is set
if settings.DEBUG:
    import debug_toolbar
    urlpatterns = [
        path('__debug__', include(debug_toolbar.urls)),
    ] + urlpatterns

# 3f2540e4b1bf996c396e04c5463a47e42900441a
```

Example

djangoexamples/contacts/contacts_core/urls.py

```
"""
URL Configuration for contacts_core
"""
from django.urls import path, include

app_name = 'contacts_core'

urlpatterns = [
    path('api/', include('contacts_core.api.urls', )),
]
```


© 2026 JS Consulting *(Licensed to Webucator)*

Example

djangoexamples/contacts/contacts_core/api/urls.py

```
from django.urls import path, include
from rest_framework import routers

from . import fb_views
from . import cb_views
from . import viewsets

app_name = 'api'

router = routers.DefaultRouter()
router.register('contacts', viewsets.ContactViewSet)
router.register('cities', viewsets.CityViewSet)

urlpatterns = [
    # path('fbv/hello', fb_views.hello, name="hello"),
    path('fbv/hello', fb_views.hello, name="hello"),
    path('fbv/contacts', fb_views.contacts, name="fbcontacts"),
    path('fbv/contacts/<str:pk>', fb_views.contacts_detail, name="fbcontacts-detail"),
    path('cbv/contacts', cb_views.ContactsList.as_view(), name="contacts"),
    path('cbv/contacts/<str:pk>', cb_views.ContactsDetail.as_view(), name="cbcontacts-
detail"),
#    path('cbv/contactsx/<str:pk>', cb_views.ContactsList.as_view(), name="cbcontacts-
detail"),
    path('cbv/cities', cb_views.CitiesList.as_view(), name="cities"),
    path('cbv/cities/<str:pk>', cb_views.CitiesDetail.as_view(), name="cbcities-detail"),
    path('', include(router.urls)),
]
```

Class-based Views

- Builtin view functions
- Just need object and serializer
- Take care of rendering

While you *can* create function-based views, most developers use either viewsets or class-based views. We'll take a look at class-based views (CBVs) here, and viewsets get their own chapter.

To create a class-based view, import the app's models, and import classes from rest_framework.generics.

All that's needed for simple cases is to specify the queryset — the list of objects that the class represents, and the serializer class for those objects.

To use the class-based view, add it to a URL config. Since the URL config needs a *callable* (normally a function or method), CBVs have a method `as_view()` which returns a callable object that will implement the view.

Example URL config

```
path('cbv/contacts/<str:pk>', cb_views.ContactsDetail.as_view(), name="cbcontacts-
detail"),
```

 © 2026 JS Consulting *(Licensed to Webucator)*

Example

djangoexamples/contacts/contacts_core/api/cb_views.py

```
# not needed for REST CBVs or Viewsets
from contacts_core.models import Contact, City
from rest_framework import generics
from  .serializers import ContactSerializer, CitySerializer

# class-based views (aka CBVs)
class ContactsList(generics.ListCreateAPIView):   #  GET /api/contacts  POST
/api/contacts

    queryset = Contact.objects.all()
    serializer_class = ContactSerializer

class ContactsDetail(generics.RetrieveUpdateDestroyAPIView):
    # GET /api/contacts/ID  PUT /api/contacts/ID  PATCH /api/contacts/ID  DELETE
/api/contacts/ID
    queryset = Contact.objects.all()
    serializer_class = ContactSerializer

class CitiesList(generics.ListCreateAPIView):
    queryset = City.objects.all()
    serializer_class = CitySerializer

class CitiesDetail(generics.RetrieveUpdateDestroyAPIView):
    queryset = City.objects.all()
    serializer_class = CitySerializer
```

Chapter 1 Exercises

Exercise 1-1: (dogs/dogs_core/*)

Create a simple (non model-based) serializer for the Dog model, and a simple function-based view to display it.

Exercise 1-2: (dogs/dogs_core/*)

Create a model-based serializer for the Dog model, and a class-based view to display it.

© 2026 JS Consulting *(Licensed to Webucator)*

Chapter 2: Django REST Viewsets

Objectives

- Understand viewsets
- Expose models using viewsets
- Provide pagination

What are viewsets?

- One viewset represents one resource
- Higher level of abstraction
- Tools for exposing models
- Less coding than class-based views

viewsets are REST Framework classes that go beyond class-based views to a higher level of abstraction. A viewset completely represents one resource.

They provide methods such as `list()` and `create()` rather than mapping directly to the HTTP verbs.

In practice, they mean that you have even less code to write. The combine all the logic for the views for a model in a single class.

Another advantage of viewsets is that they define their own routes, so you typically only have to add one line to your URL config for each model, and don't have to worry about naming the routes individually.

In terms of class-based views, a viewset would be a combination of `ListCreateAPIView` and `RetrieveUpdateDestroyAPIView`.

 © 2026 JS Consulting *(Licensed to Webucator)*

Creating Viewsets

- Import `viewsets`
- Define class
- Specify queryset and serializer

In many ways, using a viewset is similar to using a class-based view. You import a viewset, create a class to subclass it, and specify the model and serializer needed.

There are serveral base viewset classes. Which one to choose depends on how much customization there is. The most generic viewset is ViewSet, which requires you to write your own methods as needed.

The most commonly used viewset is ModelViewset, which exposes the data from a model, using the model's serializer.

you can customize viewsets in the same ways you can customize class-based views.

Example

djangoexamples/contacts/contacts_core/api/viewsets.py

```
from rest_framework import viewsets
from .serializers import ContactSerializer, CitySerializer
#  from contacts_core.api.filters import * # (optional)  change to only needed
serializers
from ..models import Contact, City

class ContactViewSet(viewsets.ModelViewSet):
    queryset = Contact.objects.all()
    serializer_class = ContactSerializer
    # filter_backends = [DjangoFilterBackend]
    # filterset_class = MyFirstModelFilter
    # pagination_class = ...


class CityViewSet(viewsets.ModelViewSet):
    queryset = City.objects.all()
    serializer_class = CitySerializer
    # filter_backends = [DjangoFilterBackend]
    # filterset_class = MySecondModelFilter
```

Table 1. Available ViewSets

Viewset	Decription
`ViewSet`	Minimal viewset — add attributes (auth classes, permission classes) to define behavior
`GenericViewSet`	Inherits from GenericAPIView, but doesn't define actions
`ModelViewSet`	Includes implementations of standard actions.
`ReadOnlyModelViewSet`	like ModelViewSet, but read-only

 © 2026 JS Consulting *(Licensed to Webucator)*

Setting up routes

- Create a **router**
- Register viewsets with routers
- Add router.urls to URL config

To set up routes for a viewset, import **routers** from `rest_framework`.

Create a DefaultRouter (or other router) and register one or more viewsets with it. The name a viewset is registered with is incorporated into its route. These names are the resource endpoints.

In `urlpatterns`, use `include()` to delegate to the router-generated urls.

Routers, like everything else in Django, can be customized.

Example

djangoexamples/contacts/contacts_core/api/urls.py

```
from django.urls import path, include
from rest_framework import routers

from . import fb_views
from . import cb_views
from . import viewsets

app_name = 'api'

router = routers.DefaultRouter()
router.register('contacts', viewsets.ContactViewSet)
router.register('cities', viewsets.CityViewSet)

urlpatterns = [
    # path('fbv/hello', fb_views.hello, name="hello"),
    path('fbv/hello', fb_views.hello, name="hello"),
    path('fbv/contacts', fb_views.contacts, name="fbcontacts"),
    path('fbv/contacts/<str:pk>', fb_views.contacts_detail, name="fbcontacts-detail"),
    path('cbv/contacts', cb_views.ContactsList.as_view(), name="contacts"),
    path('cbv/contacts/<str:pk>', cb_views.ContactsDetail.as_view(), name="cbcontacts-
detail"),
#    path('cbv/contactsx/<str:pk>', cb_views.ContactsList.as_view(), name="cbcontacts-
detail"),
    path('cbv/cities', cb_views.CitiesList.as_view(), name="cities"),
    path('cbv/cities/<str:pk>', cb_views.CitiesDetail.as_view(), name="cbcities-detail"),
    path('', include(router.urls)),
]
```


© 2026 JS Consulting *(Licensed to Webucator)*

Customizing viewsets

- Add class-level attributes
- Useful attributes
  - permission classes
  - lookup_field
  - pagination_class
  - filter_backends

In addition to `queryset` and `serializer_class`, there are many attributes than can be defined on a viewset.

`permission_classes` allows you to specify what permissions are required to access data in the view.

`lookup_field` specifies a field other than **pk** (the default) for item lookup.

`pagination_class` specifies a class for controlling pagination.

`filter_backends` provides filters as HTTP query strings to search the models.

See https://www.django-rest-framework.org/api-guide/generic-views/#genericapiview for a complete list of available attributes.

Adding pagination

- Controls number of items retrieved
- Can be set
  - App-wide
  - Per model

If your database has ten million **wombat** records and you make a generic request to list the **wombat** resource, it may overwhelm your client application. A well-designed API should allow limits and pagination.

REST Framework provides the **LimitOffsetPagination** class, which lets you provide a limit and an offset. The offset is the (0-based) first object to retrieve, and limit is the number of items to retrieve.

Also, using pagination adds `next` and `previous` fields to your result so that a client app can use them for navigation.

Normally, you want all views to have the same pagination, so you set it up in `settings.py` as one of the entries in REST_FRAMEWORK:

```
'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.LimitOffsetPagination',
```

Assign to the `pagination_class` attribute of a class-based view or viewset if you want to change the pagination scheme for that particular model.

set `pagination_class=None` to disable pagination.

© 2026 JS Consulting *(Licensed to Webucator)*

Chapter 2 Exercises

Exercise 2-1 (dogs/dogs_core/*)

Add viewsets to your bands API.

(Optional): Add pagination.

Chapter 3: Django REST Filters

Objectives

- Design filters to enable query strings
- Create shortcuts for filters
- Use filters with viewsets or class-based views

© 2026 JS Consulting *(Licensed to Webucator)*

What are filters?

- Automate querying of models
- Allow query strings on URLs
- For REST
  - Search records
  - Paginate (limit number of records)

Filters are classes that automate querying of models. Rather than having to write numerous `MyModel.objects.filter(…)` statement in hand-crafted viewset methods, you can specify which field you want to search and how you want to search them.

Filters can be used with both class-based views and viewsets.

For the details on creating filters, see https://django-filter.readthedocs.io/en/stable/guide/usage.html

Creating filters

- Import **rest_framework** from django_filters
  - Convenient to alias as **filters**
- Subclass **filters.FilterSet**
- Define filters

For each model create a *filteret* by subclassing **rest_framework.django_filters.FilterSet**.

In the filterset, add a filter for each field you want to filter. You can add more than one filter for a field. for instance, for a year field you could have **min_year**, **max_year**, and **year**. You can be creative — you could add **before_1950** as a filter.

Each filter is a filter type from **django_filters.rest_framework**. The name of the filter becomes the querystring on the URL for GET requests. Choose short, self-explanatory filter names, since these will be used by the clients.

Each filter must be listed in the **fields** attribute of the **Meta** class within the filterset. You also need to specify the model to be filtered in **Meta.model**.

For related queries, use `<related-field>__<relation>`.

 © 2026 JS Consulting *(Licensed to Webucator)*

Example

djangoexamples/contacts/contacts_core/api/filters.py

```
from django_filters import rest_framework as filters
from contacts_core.models import *  # change to only required models

#  class MyFirstModelFilter(filters.FilterSet):
#      name = filters.CharFilter(field_name="name", lookup_expr='icontains')
#      birthplace = filters.CharFilter(field_name="place_of_birth",
lookup_expr='icontains')
#
#      """MAX BIRTH YEAR"""
#      min_birth_year = filters.NumberFilter(field_name="birth_year", lookup_expr='gte')
#
#
#      max_birth_year = filters.NumberFilter(field_name="birth_year", lookup_expr='lte')
#
#      min_death_year = filters.NumberFilter(field_name="death_year", lookup_expr='gte')
#      max_death_year = filters.NumberFilter(field_name="death_year", lookup_expr='lte')
#
#      alive = filters.BooleanFilter(field_name="death_year", lookup_expr='isnull')
#
#
#      class Meta:
#          model = MyFirstModel
#          fields = ["field1", "field2,]  # ...
#
#  class MySecondModelFilter(filters.FilterSet):
#      title = filters.CharFilter(field_name="title", lookup_expr='icontains')
#
#      class Meta:
#          model = MySecondModel
#          fields = ["field1", "field2",]  # ...
```

Filter shortcuts

- Auto-generates **equal** filter
- Use dictionary for flexibility
- Don’t need to define individual filters.

If a field is listed in **Meta.fields**, but there is no filter defined, an "equal" filter will be created.

```
class ContactFilter(filters.FilterSet):
     class Meta:
         model = Contact
         fields = ["first_name", "last_name"]
```

If you want to use a different relation, **Meta.fields** can be a dictionary, where the key is the filter name, and the value is a list of relations:

```
class ContactFilter(filters.FilterSet):
    class Meta:
        model = Contact
        fields = {"first_name": ['icontains'], "last_name": ['icontains"]}
```


© 2026 JS Consulting *(Licensed to Webucator)*

Using filters

- Specify in CBVs or viewsets
- Use query strings in URLs

To use filters in either class-based views or viewsets, add the following two fields:

filter_backend set to **django_filters.rest_framework.DjangoFilterBackend**

filterset_class set to the filterset class you defined.

Example

djangoexamples/contacts/contacts_core/api/viewsets.py

```
from rest_framework import viewsets
from .serializers import ContactSerializer, CitySerializer
#  from contacts_core.api.filters import * # (optional)  change to only needed
serializers
from ..models import Contact, City

class ContactViewSet(viewsets.ModelViewSet):
    queryset = Contact.objects.all()
    serializer_class = ContactSerializer
    # filter_backends = [DjangoFilterBackend]
    # filterset_class = MyFirstModelFilter
    # pagination_class = ...


class CityViewSet(viewsets.ModelViewSet):
    queryset = City.objects.all()
    serializer_class = CitySerializer
    # filter_backends = [DjangoFilterBackend]
    # filterset_class = MySecondModelFilter
```

Chapter 3 Exercises

Exercise 3-1 (labname.py)

Add filters to the **dogs** project. Clients should be able to filter on the **name**, **weight**, **breed**, and **sex** fields of **dog** objects, and on the **name** field of **breed** objects.

Run your project and try out the querystrings.

© 2026 JS Consulting *(Licensed to Webucator)*

Chapter 4: Django REST Documentation

Objectives

- Generate browsable API docs
- Generate OpenAPI schemas

Documentation?

- Useful before coding
- Useful after coding
- Critical for client apps

Of course, it is important to provide documentation for applications. In the case of REST, it is crucial for anyone writing client code for your REST endpoints.

You can generate a schema from your existing API using various tools.

© 2026 JS Consulting *(Licensed to Webucator)*

Generating an OpenAPI schema: Part I

- Use `pm generateschema
- Choice of formats

One way to generate a schema is to use the **generateschema** command that is part of **manage.py**. This generates a schema in YAML or JSON formats, with options to fine-tune the generation.

```
manage.py generateschema --format openapi --file myschema.yaml
```

Generating an OpenAPI schema: Part II

- Install **drf-spectactular**
- Add routes

To generate an OpenAPI schema, you can install the **drf-spectacular** extension to the REST Framework.

Then you just need to add routes to your URL config.

To install, use

```
pip install drf-spectacular
```

Add the following to **REST_FRAMEWORK** in `settings.py`.

```
'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
```

Then, in your project-level URL config, import views from `drf_spectacular.views`.

You can define route the three views as follows:

```
path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
# Optional UI:
path('api/schema/swagger-ui/', SpectacularSwaggerView.as_view(url_name='schema'),
name='swagger-ui'),
path('api/schema/redoc/', SpectacularRedocView.as_view(url_name='schema'), name='redoc'),
]
```

Now just visit the defined URLs.

- api/schema *download the schema as YAML*
- api/schema/swagger-ui *browsable Swagger/OpenAPI schema*
- api/schema/redoc *browsable ReDoc schema*

The ReDoc schema grew out of the Swagger schema, and provides a 3-panel layout.

© 2026 JS Consulting *(Licensed to Webucator)*

Chapter 4 Exercises

Exercise 4-1 (dogs/dogs/*)

Use drf-spectacular to generate the OpenAPI spec for the dogs API.

Generate and test Python client code.

Chapter 5: Unit Testing with pytest

Objectives

- Understand the purpose of unit tests
- Design and implement unit tests with pytest
- Run tests in different ways
- Use builtin fixtures
- Create and use custom fixtures
- Mark tests for running in groups
- Learn how to mock data for tests

© 2026 JS Consulting *(Licensed to Webucator)*

What is a unit test?

- Tests *unit* of code in isolation
- Ensures repeatable results
- Asserts expected behavior

A *unit test* is a test which asserts that an isolated piece of code (one function, method, class, or module) has some expected behavior. It is a way of making sure that code provides repeatable results.

There are four main components of a unit testing system:

1. Unit tests – individual assertions that an expected condition has been met
2. Test cases – collections of related unit tests
3. Fixtures — provide data to set up tests in order to get repeatable results
4. Test runners – utilities to execute the tests in one or more test cases

Unit tests should each test one aspect of your code, and each test should be independent of all other tests, including the order in which tests are run.

Each test asserts that some condition is true.

Unit tests may collected into a **test case**, which is a related group of unit tests. With `pytest`, a test case can be either a module or a class.

Fixtures provide repeatable, known input to a test.

The final component is a **Test runner**, which executes one, some, or all tests and reports on the results. There are many different test runners for pytest. The builtin runner is very flexible.

The pytest module

- Provides
  - test runner
  - fixtures
  - special assertions
  - extra tools
- Not based on xUnit[1]

The `pytest` module provides tools for creating, running, and managing unit tests.

Here's how `pytest` implements the main components of unit testing:

unit test

A normal Python function that uses the `assert` statement to assert some condition is true

test case

A class that contains a group of related unit tests (tests can be also be grouped with *markers*).

fixture

A function that provides test resources (fixtures can be nested).

test runner

A text-based test runner is built in, and there are many third-party test runners

`pytest` is more flexible than classic **xUnit** implementations. For example, fixtures can be associated with any number of individual tests, or with a test class. Test cases need not be classes. These features help make it popular.

[1] The builtin unit testing module, `unittest`, *is* based on **xUnit** patterns, as implemented in Java and other languages. The `pytest` test runner will detect `unittest` tests as well. This can be handy for transitioning legacy code to `pytest`.

© 2026 JS Consulting *(Licensed to Webucator)*

Installing the modules to test

Before running some tests, `spamlib` and `hamlib` must be locally installed on the computer where you're taking the class; otherwise the tests won't be able to import them. You can install them in editable mode like this:

From the EXAMPLES folder:

```
cd hamlib
pip install -e .

cd ../spamlib
pip install -e .
```

Creating tests

- Create test functions
- Use builtin `assert`
- Confirm something is true
- Optional message

To create a test, create a function whose name begins with "test". These should normally be in a separate script, whose name begins with "test_" or ends with "_test". For the simplest cases, tests do not even need to import `pytest`.

Each test function will use the builtin `assert` statement one or more times to confirm that that some condition is true. If all of the assertions are True, then the test passes. If any assertion fails, the test fails.

`pytest` will print an appropriate message by introspecting the expression, or you can add your own failure message after the expression, separated by a comma.

```
assert result == 100
assert 2 == 2, "Two is not equal to two!"
```

It is a good idea to make test names verbose. This will help when running tests in verbose mode, so you can see what tests are passing (or failing).

Real-life unit tests

`requests` is one of the most commonly used Python modules outside of the standard library. It provides an HTTP client with many helpful options. Here are some of the unit tests for `requests`:

https://github.com/psf/requests/blob/main/tests/test_requests.py

© 2026 JS Consulting *(Licensed to Webucator)*

Example

tests/test_simple.py

```
# tests should begin with "test" for automatic discovery
def test_two_plus_two_equals_four():
    assert 2 + 2 == 4   # if assert statement succeeds, the test passes

def test_three_is_greater_than_two():
    assert 3 > 2
```

pytest -v tests/test_simple.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 2 items

tests/test_simple.py::test_two_plus_two_equals_four PASSED               [ 50%]
tests/test_simple.py::test_three_is_greater_than_two PASSED              [100%]

============================== 2 passed in 0.20s ===============================
```

Running tests (basics)

- Needs a test runner
- `pytest` provides `pytest` script

To actually run tests, you need a *test runner*. A test runner is software that runs one or more tests and reports the results.

`pytest` provides a script (also named `pytest`) to run tests from the command line (if using Anaconda, do this from an Anaconda prompt).

You can run a single test, a test case, a module, or all tests in a folder and all its subfolders.

```
pytest test_⋯py
```

to run the tests in a particular module, and

```
pytest -v test_⋯py
```

to add verbose output.

By default, pytest captures (and does not display) anything written to stdout/stderr. If you want to see the output of `print()` statements in your tests, add the **-s** option, which turns off output capture.

```
pytest -s ⋯
```

If pytest does not run from the command line, use `python -m pytest`.

In older versions of pytest, the test runner script was named `py.test`. While newer versions support that name, the developers recommend only using `pytest`.

© 2026 JS Consulting *(Licensed to Webucator)*

Special assertions

- Special cases
  - pytest.raises()
  - pytest.approx()

There are two special cases not easily handled by `assert`.

pytest.raises

For testing whether an exception is raised, use `pytest.raises()`. This should be used with the **with** statement:

```
with pytest.raises(ValueError):
    w = Wombat('blah')
```

The assertion will succeed if the code inside the **with** block raises the specified error.

pytest.approx

For testing whether two floating point numbers are *close enough* to each other, use `pytest.approx()`:

```
assert result == pytest.approx(1.55)
```

The default tolerance is 1e-6 (one part in a million). You can specify the relative or absolute tolerance to any degree. Infinity and NaN are special cases. NaN is normally not equal to anything, even itself, but you can specify `nanok=True` as an argument to approx().

See https://docs.pytest.org/en/latest/reference.html#pytest-approx for more information on pytest.approx()

Example

tests/test_special_assertions.py

```
import pytest
import math

FILE_NAME = 'IDONOTEXIST.txt'

#########################################################
# subject under test                                    #
#########################################################
def read_file_data(file_name):
    with open(file_name) as file_in:
        data = file_in.read().splitlines()
        return data
#########################################################


def test_missing_filename():
    """
    Assert FileNotFoundError is raised
    """
    with pytest.raises(FileNotFoundError):
        read_file_data(FILE_NAME)  # will pass test if file is NOT found


def test_floats_approximately_match():
    # fail unless values are within 0.000001 of each other
    # (actual result is 0.30000000000000004)
    assert (.1 + .2) == pytest.approx(.3)


def test_22_div_7_is_approximately_pi():
    # Default tolerance is 0.000001
    # smaller (or larger) tolerance can be specified
    assert 22 / 7 == pytest.approx(math.pi, .001)
```


© 2026 JS Consulting *(Licensed to Webucator)*

pytest -v tests/test_special_assertions.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 3 items

tests/test_special_assertions.py::test_missing_filename PASSED           [ 33%]
tests/test_special_assertions.py::test_floats_approximately_match PASSED [ 66%]
tests/test_special_assertions.py::test_22_div_7_is_approximately_pi PASSED [100%]

============================== 3 passed in 0.17s ===============================
```

Fixtures

- Provide resources for tests
- Implement as functions
- Scope
  - Per test
  - Per class
  - Per module
- Source of fixtures
  - Builtin
  - User-defined

When writing tests for a particular object, many tests might require an instance of the object. This instance might be created with a particular set of arguments.

What happens if twenty different tests instantiate a particular object, and the object's API changes? Now you have to make changes in twenty different places.

To avoid duplicating code across many tests, pytest supports *fixtures*, which are functions that provide information to tests. The same fixture can be used by many tests, which lets you keep the fixture creation in a single place.

A fixture provides items needed by a test, such as data, functions, or class instances. A fixtures can be either builtin or custom.

What fixtures provide

Consistency

test uses the same, repeatable data

Readability

keeps test itself short and simple

Auto-use

Reduces number of imports

Teardown

Provides cleanup capabilities

Use `pytest --fixtures` to list all available builtin and user-defined fixtures.

© 2026 JS Consulting *(Licensed to Webucator)*

User-defined fixtures

- Decorate with **pytest.fixture**
- Return value to be used in test
- Fixtures may be nested

To create a fixture, decorate a function with `pytest.fixture`. Whatever the function returns is the value of the fixture.

To use the fixture, pass it to the test function as a parameter. The return value of the fixture will be available as a local variable in the test.

Fixtures can take other fixtures as parameters as well, so they can be nested to any level.

It is convenient to put fixtures into a separate module so they can be shared across multiple test scripts.

Add docstrings to your fixtures and the docstrings will be displayed via `pytest --fixtures`

Example

tests/test_simple_fixture.py

```
import os
import pytest

from .silly import Silly

@pytest.fixture
def silly_object():
    return Silly()  # fixture returns instance of Silly

def test_silly_triples_value(silly_object):  # pass fixture as test parameter
    assert silly_object.triple(5) == 15

def test_silly_normalizes_string(silly_object):
    assert silly_object.normalize("   Spam   ") == "spam"
```


© 2026 JS Consulting *(Licensed to Webucator)*

pytest -v tests/test_simple_fixture.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 2 items

tests/test_simple_fixture.py::test_silly_triples_value PASSED            [ 50%]
tests/test_simple_fixture.py::test_silly_normalizes_string PASSED        [100%]

============================== 2 passed in 0.16s ===============================
```

Builtin fixtures

- Variety of common fixtures
- Provide
  - Temp files and dirs
  - Logging
  - STDOUT/STDERR capture
  - Monkeypatching tools

Pytest provides a large number of builtin fixtures for common testing requirements.

Using a builtin fixture is like using user-defined fixtures. Just specify the fixture name as a parameter to the test. No imports are needed for this.

See https://docs.pytest.org/en/latest/reference.html#fixtures for details on builtin fixtures.

© 2026 JS Consulting *(Licensed to Webucator)*

Example

tests/test_builtin_fixtures.py

```
import pytest

COUNTER_KEY = 'test_cache/counter'

def test_cache(cache):  # cache persists values between test runs
    value = cache.get(COUNTER_KEY, 0)
    print("\nCounter before:", value)
    cache.set(COUNTER_KEY, value + 1)  # cache fixture is similar to dictionary, but with
.set() and .get() methods
    value = cache.get(COUNTER_KEY, 0)  # cache fixture is similar to dictionary, but with
.set() and .get() methods
    print("\nCounter after:", value)
    assert True   # Make test successful

def hello():
    print("Hello, pytesting world")

def test_capsys(capsys):
    hello()  # Call function that writes text to STDOUT
    out, err = capsys.readouterr()  # Get captured output
    assert out == "Hello, pytesting world\n"

@pytest.mark.parametrize("num", range(3))
def test_temp_dir(tmpdir, num):
    print("TEMP DIR:", tmpdir, "\n")  # tmpdir fixture provides unique temporary folder
name
```

pytest -v tests/test_builtin_fixtures.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 5 items

tests/test_builtin_fixtures.py::test_cache PASSED                        [ 20%]
tests/test_builtin_fixtures.py::test_capsys PASSED                       [ 40%]
tests/test_builtin_fixtures.py::test_temp_dir[0] PASSED                  [ 60%]
tests/test_builtin_fixtures.py::test_temp_dir[1] PASSED                  [ 80%]
tests/test_builtin_fixtures.py::test_temp_dir[2] PASSED                  [100%]
```

```
============================== 5 passed in 0.18s ==============================
```


© 2026 JS Consulting *(Licensed to Webucator)*

Table 2. Pytest Builtin Fixtures

Fixture	Brief Description
`cache`	Return cache object to persist state between testing sessions.
`capsys`	Enable capturing of writes (text mode) to `sys.stdout` and `sys.stderr`
`capsysbinary`	Enable capturing of writes (binary mode) to `sys.stdout` and `sys.stderr`
`capfd`	Enable capturing of writes (text mode) to file descriptors 1 and 2
`capfdbinary`	Enable capturing of writes (binary mode) to file descriptors 1 and 2
`doctest_namespace`	Return `dict` that will be injected into namespace of doctests
`pytestconfig`	Session-scoped fixture that returns `_pytest.config.Config` object.
`record_property`	Add extra properties to the calling test.
`record_xml_attribute`	Add extra xml attributes to the tag for the calling test.
`caplog`	Access and control log capturing.
`monkeypatch`	Return `monkeypatch` fixture providing monkeypatching tools
`recwarn`	Return `WarningsRecorder` instance that records all warnings emitted by test functions.
`tmp_path`	Return `pathlib.Path` instance with unique temp directory
`tmp_path_factory`	Return a `_pytest.tmpdir.TempPathFactory` instance for the test session.
`tmpdir`	Return `py.path.local` instance unique to each test
`tmpdir_factory`	Return `TempdirFactory` instance for the test session.

Configuring fixtures

- Create **conftest.py**
- Automatically included
- Provides
  - Fixtures
  - Hooks
  - Plugins
- Directory scope

The `conftest.py` file can be used to contain user-defined fixtures, as well as hooks and plugins. Subfolders can have their own conftest.py, which will only apply to tests in that folder.

In a test folder, define one or more fixtures in conftest.py, and they will be available to all tests in that folder, as well as any subfolders.

Hooks

Hooks are predefined functions that will automatically be called at various points in testing. All hooks start with *pytest_*. A pytest.Function object, which contains the actual test function, is passed into the hook.

For instance, `pytest_runtest_setup()` will be called before each test.

> A complete list of hooks can be found here: https://docs.pytest.org/en/latest/reference.html#hooks

Plugins

There are many pytest plugins to provide helpers for testing code that uses common libraries, such as **Django** or **redis**.

You can register plugins in conftest.py like so:

```
pytest_plugins = "plugin1", "plugin2",
```

This will load the plugins.

© 2026 JS Consulting *(Licensed to Webucator)*

Example

tests/conftest.py

```
#!/usr/bin/env python
from pytest import fixture
from .silly import Silly

@fixture
def silly_object():
    return Silly()  # fixture returns instance of Silly

# predefined hook (all hooks start with 'pytest_')
def pytest_runtest_setup(item):  # item is test function
    if "test_config" in str(item):
        print(f"Hello from setup, {item}", end=" ")
```

Example

tests/test_config.py

```
def test_one(silly_object):   # unit test that uses fixture from conftest.py
    assert silly_object.triple(5) == 15
    assert silly_object.normalize("   Spam   ") == "spam"
```

pytest -v tests/test_config.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 1 item

tests/test_config.py::test_one PASSED                                    [100%]

============================== 1 passed in 0.17s ===============================
```


© 2026 JS Consulting *(Licensed to Webucator)*

Parametrizing tests

- Run same test on multiple values
- Add parameters to fixture decorator
- Test run once for each parameter
- Use `pytest.mark.parametrize()`

Many tests require testing a method or function against many values. Rather than writing a loop in the test, you can automatically repeat the test for a set of inputs via **parametrizing**.

Apply the `@pytest.mark.parametrize` decorator to the test. The first argument is a string with the comma-separated names of the parameters; the second argument is the list of parameters. The test will be called once for each item in the parameter list. If a parameter list item is a tuple or other multi-value object, the items will be passed to the test based on the names in the first argument.

For more advanced needs, when you need some extra work to be done before the test, you can do indirect parametrizing, which uses a parametrized fixture. See `test_parametrize_indirect.py` for an example.

The authors of pytest deliberately spelled it "parametrizing", not "parameterizing".

Example

tests/test_parametrization.py

```
import pytest
from .silly import Silly

@pytest.fixture
def silly_object():
    return Silly()  # fixture returns instance of Silly

# List of values for testing containing input and expected result
test_data = [
    (5, 15), ('a', 'aaa'), (False, False), (True, 3), (0, 0), ([True], [True, True,
True])
]

# Parametrize the test with the test data; the first argument is a string mapping
# parameters to the test data
@pytest.mark.parametrize("input,expected", test_data)
def test_triple(silly_object, input, expected):
    assert silly_object.triple(input) == expected   # Test the function with the
parameters
```


© 2026 JS Consulting *(Licensed to Webucator)*

pytest -v tests/test_parametrization.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 6 items

tests/test_parametrization.py::test_triple[5-15] PASSED                  [ 16%]
tests/test_parametrization.py::test_triple[a-aaa] PASSED                 [ 33%]
tests/test_parametrization.py::test_triple[False-False] PASSED           [ 50%]
tests/test_parametrization.py::test_triple[True-3] PASSED                [ 66%]
tests/test_parametrization.py::test_triple[0-0] PASSED                   [ 83%]
tests/test_parametrization.py::test_triple[input5-expected5] PASSED      [100%]

============================== 6 passed in 0.18s ===============================
```

Marking tests

- Create groups of tests ("test cases")
- Can create multiple groups
- Use `@pytest.mark.somemark`

You can mark tests with labels so that they can be run as a group. Use `@pytest.mark.marker`, where *marker* is the marker (label), which can be any alphanumeric string.

Then you can select tests which contain or match the marker.

```
pytest -m "alpha"
pytest -m "not alpha"
pytest -m "alpha or beta"
pytest -m "alpha and not beta"
```

Registering markers

You can register markers in the **[pytest]** section of `pytest.ini`, so they will be listed, with a description, with `pytest --markers`:

```
[pytest]
markers =
  internet: test requires internet connection
  slow: tests that take more time (omit with '-m "not slow")
```


© 2026 JS Consulting *(Licensed to Webucator)*

Example

tests/test_mark.py

```
import pytest

@pytest.mark.alpha  # Mark with label alpha
def test_one():
    assert 1

@pytest.mark.beta
@pytest.mark.alpha  # Mark with label alpha
def test_two():
    assert 1

@pytest.mark.gamma
@pytest.mark.beta  # Mark with label beta
def test_three():
    assert 1
```

pytest -v tests/test_mark.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 3 items

tests/test_mark.py::test_one PASSED                                      [ 33%]
tests/test_mark.py::test_two PASSED                                      [ 66%]
tests/test_mark.py::test_three PASSED                                    [100%]

============================== 3 passed in 0.17s ===============================
```

Running tests (advanced)

- Run all tests
- Run by
  - file name
  - class
  - function
  - name match
  - group (mark)

`pytest` provides many ways to select which tests to run.

Running all tests

By default, pytest will find and run all tests in the current and any descendent directories.

Use `-s` to disable capturing, so anything written to STDOUT is displayed. Use `-v` for verbose output.

```
pytest
pytest -v
pytest -s
pytest -vs
```

Running by component

Use the node ID to select by component, such as file, class, method, or function name:

```
file::class
file::class::test
file::test
```

```
pytest test_president.py::test_dates
pytest test_president.py::test_dates::test_birth_date
```

Note that the *file* name is specified, not the *module* name.

© 2026 JS Consulting *(Licensed to Webucator)*

Running by name match

Use **-k** to run all tests where the file name, test name, or marker includes a specified string.

`pytest -k date` *run all tests whose name includes 'date'*

Making test files executable

While you should normally use `pytest` or your IDE to run tests, you can also make a test script run the tests when you execute the script normally with `python`. To do this, put the following code at the bottom of the test file:

```
if __name__ == '__main__':
    pytest.main([__file__, '-v'])   # Start the test runner
```

`__file__` is the name of the current file. You can add the '-s' option as another element in the list of arguments to `pytest.main`. You can also omit the '-v' option if you don't want verbose output.

Most of the time you should just use the test runner.

© 2026 JS Consulting *(Licensed to Webucator)*

Skipping and failing

- Conditionally skip tests
- Completely ignore tests
- Decorate with
  - @pytest.mark.xfail
  - @pytest.mark.skip

To skip tests conditionally (or unconditionally), use `@pytest.mark.skip()`. This is useful if some tests rely on components that haven't been developed yet, or for tests that are platform-specific.

To fail on purpose, use `@pytest.mark.xfail)`. This reports the test as "XPASS" or "xfail", but does not provide traceback. Tests marked with xfail will not fail the test suite. This is useful for writing tests for not-yet-implemented features, or for testing objects with known bugs that will be resolved later.

Example

tests/test_skip_xfail.py

```
import sys
import pytest

def test_one():  # Normal test -- no skipping or failing
    assert True

# Uncondition skip
@pytest.mark.skip(reason="can not currently test")
def test_two():
    assert True

# Skip this test if current platform is not Windows
@pytest.mark.skipif(
    sys.platform != 'win32',
    reason="only implemented on Windows"
)
def test_three():
    assert True

@pytest.mark.xfail
def test_four():
    assert False

@pytest.mark.xfail
def test_five():
    assert True
```


© 2026 JS Consulting *(Licensed to Webucator)*

pytest -v tests/test_skip_xfail.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 5 items

tests/test_skip_xfail.py::test_one PASSED                                [ 20%]
tests/test_skip_xfail.py::test_two SKIPPED (can not currently test)      [ 40%]
tests/test_skip_xfail.py::test_three SKIPPED (only implemented on Wi...) [ 60%]
tests/test_skip_xfail.py::test_four XFAIL                                [ 80%]
tests/test_skip_xfail.py::test_five XPASS                                [100%]

============== 1 passed, 2 skipped, 1 xfailed, 1 xpassed in 0.24s ==============
```

Mocking data

- Simulate behavior of actual objects
- Replace expensive dependencies (time/resources)
- Use **pytest-mock**

Some objects have dependencies which can make unit testing difficult. These dependencies may be expensive in terms of time or resources, or can be slow to access.

One solution is to create fake dependencies, called *stubs*. A stub is a function or class that acts like the actual dependency, but has hard-coded data and doesn't necessarily implement all features of the dependency.

A problem with stubs is that you end up with a lot of single-purpose stubs, which can be hard to maintain.

The better solution is to use **mock** objects, which pretend to be the real object. A mock object behaves like the original object, but is restricted and controlled in its behavior.

For instance, a class may have a dependency on a database query. A mock object may accept the query, but always returns a hard-coded set of results.

A mock object can record the calls made to it, and assert that the calls were made with correct parameters.

A mock object can be preloaded with a return value, or a function that provides dynamic (or random) return values.

Unlike a stub, a mock object is more elaborate, with record/playback capability, assertions, and many other test-related features.

© 2026 JS Consulting *(Licensed to Webucator)*

Mocking in pytest

- Use `pytest-mock` plugin
  - Act as a stub
  - Patch existing code
- Can also use unittest.mock.Mock

The standard library provides `unittest.mock`. The `pytest-mock` plugin for `pytest` provides a wrapper around `unittest.mock`.

Once `pytest-mock` is installed, it provides a fixture named `mocker`, from which you can create mock objects.

In either case, there are two primary ways of mocking dependencies. One is to provide a replacement class, function, or data object that mimics the real thing.

The second is to patch a module, which temporarily (just during the test) replaces a component with a mock version. The `mocker.patch()` function replaces a component with a mock object. Any calls to the component are now recorded, and return values may be specified.

Mock where the object is used

- Don't patch original object
- Patch via subject under test

When the subject under test imports a module, it has a name that corresponds to that module. If you patch the original module, it only patches the original module, not the object pointed to by the subject under test.

For instance, if you are testing module `spam`, and it imports module `eggs` as follows, you might need to test that `spam` calls `eggs.toast`.

spam.py

```
from eggs import toast

toast()
```

Rather than patching `eggs.toast`, you will need to patch `spam.toast`. Always patch where the object is *used*, not where it's defined.

Mocking the correct name can be tricky. See https://nedbatchelder.com/blog/201908/why_your_mock_doesnt_work.html for more information.

© 2026 JS Consulting *(Licensed to Webucator)*

Example

tests/test_mock.py

```
import pytest
import spamlib
from spamlib.spam import Spam

@pytest.fixture
def ham_value():
    return 42

@pytest.fixture
def ham_result(ham_value):  # use ham_value fixture
    return ham_value * 10

def test_spam_calls_ham(mocker, ham_value, ham_result):
    # need to patch spamlib.spam.ham, not hamlib.ham
    mocker.patch("spamlib.spam.ham", return_value=ham_result)
    s = Spam(ham_value)  # Create instance of Spam, which calls ham()
    assert s.value == ham_result
    assert spamlib.spam.ham.calledoncewith(ham_value)
```

pytest -v tests/test_mock.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 0 items / 1 error

==================================== ERRORS ====================================
_________________________ ERROR collecting test_mock.py _________________________
ImportError while importing test module
'/Users/jstrick/curr/courses/python/common/examples/tests/test_mock.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/importlib/__init__.py:1
26: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^
```

```
========================== short test summary info ============================
ERROR tests/test_mock.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!
=============================== 1 error in 0.17s ==============================
```


© 2026 JS Consulting *(Licensed to Webucator)*

Example

tests/test_mock_pymock.py

```
from spamlib import spam
PATTERN = 'bug'
STRING_TO_SEARCH = 'lightning bug'

def test_spam_search_calls_re_search(mocker):   # Unit test
    # Patch re.search (i.e., replace re.search with a Mock object that
    # records calls to it)
    mocker.patch('spamlib.spam.re.search')

    s = spam.SpamSearch(PATTERN, STRING_TO_SEARCH)  # Create instance of SpamSearch
    s.findit()   # Call the method under test

    # Check that method was called just once with the expected parameters
    spam.re.search.assert_called_once_with(PATTERN, STRING_TO_SEARCH)
```

pytest -v tests/test_mock_pymock.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 0 items / 1 error

==================================== ERRORS ====================================
_____________________ ERROR collecting test_mock_pymock.py _____________________
ImportError while importing test module
'/Users/jstrick/curr/courses/python/common/examples/tests/test_mock_pymock.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/importlib/__init__.py:1
26: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^
```

```
============================== 1 error in 0.15s ==============================
```


© 2026 JS Consulting *(Licensed to Webucator)*

Example

tests/test_mock_play.py

```
import pytest
from unittest.mock import Mock

@pytest.fixture
def small_list():   # Create fixture that provides a small list
    return [1, 2, 3]


def test_m1_returns_correct_list(small_list):
    m1 = Mock(return_value=small_list)  # Create mock object that "returns" a small list
    mock_result = m1('a', 'b')  # Call mock object with arbitrary parameters
    assert mock_result == small_list  # Check the mocked result


m2 = Mock()  # Create generic mock object

m2.spam('a', 'b')  # Call fake methods on mock object
m2.ham('wombat')  # Call fake methods on mock object
m2.eggs(1, 2, 3)  # Call fake methods on mock object

print("mock calls:", m2.mock_calls)  # Mock object remembers all calls

m2.spam.assert_called_with('a', 'b')  # Assert that spam() was called with parameters 'a'
and 'b'
```

pytest -v tests/test_mock_play.py

```
============================= test session starts ==============================
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 1 item

tests/test_mock_play.py::test_m1_returns_correct_list PASSED             [100%]

============================== 1 passed in 0.17s ===============================
```


© 2026 JS Consulting *(Licensed to Webucator)*

Pytest plugins

- Common plugins
  - `pytest-qt`
  - `pytest-django`

There are some plugins for `pytest` that that integrate various frameworks which would otherwise be difficult to test directly.

The **pytest-qt** plugin provides a `qtbot` fixture that can attach widgets and invoke events. This makes it simpler to test your custom widgets.

The **pytest-django** plugin allows you to run Django with `pytest`-style tests rather than the default **unittest** style.

See https://docs.pytest.org/en/latest/reference/plugin_list.html for a complete list of plugins. There are currently 880 plugins!

Chapter 5 Exercises

Exercise 5-1 (test_president.py)

Using `pytest`, Create some unit tests for the President class you created earlier.[1]

Suggestions for tests:

- Create President objects for all current term numbers (1-*n*)
- What happens when an out-of-range term number is given?
- President 1's first name is "George"
- All presidential terms match the correct last name (use list of last names and **parametrize**)
- Confirm date fields return an object of type **datetime.date**

[1] If there was not an exercise where you created a President class, you can use **president.py** in the top-level folder of the student guide.

© 2026 JS Consulting *(Licensed to Webucator)*

Index

@

@pytest.mark.*mark*, 62

A

`assert`, 42

C

`conftest.py`, 56

D

Django, 56

E

exception, 45

F

fixtures, 39
fixtures, 48

H

hooks, 56

J

Java, 40

M

markers, 40
mock object, 70
mock objects, 70

N

node ID, 64

P

parametrizing, 59
plugins, 56
`py.test`, 44
pymock, 70
`pytest`, 40, 42
pytest
  builtin fixtures, 52
  configuring fixtures, 56
  output capture, 44
  special assertions, 45
  user-defined fixtures, 49
  verbose, 44
pytest-django, 79
`pytest-mock`, 71
pytest-qt, 79
`pytest.approx()`, 45
`pytest.fixture`, 49
`pytest.raises()`, 45

R

redis, 56
running tests, 65
  by component, 65
  by mark, 65
  by name, 65

T

test case, 39
test cases, 39
test runner, 40, 44
test runners, 39
tests
  messages, 42
tolerance
  pytest.approx, 45

U

unit test, 39
unit test components, 39
unit tests
  failing, 67
  mock objects, 71
  running, 44
  skipping, 67
unittest.mock, 70
`unittest.mock`, 71

V

viewset, 18
viewsets, 19

X

xfail, 67
XPASS, 67
xUnit, 40

© 2026 JS Consulting *(Licensed to Webucator)*